Simple Types

Concepts of Programming Languages

CAS CS 320

Outline

- Have a high-level discussion of **type theory** in general
- Introduce and analyze the **simply-typed lambda calculus** (STLC)
- Demo an **implementation** of the STLC

Recap

Recall: The Environment Model

$$\mathscr{E} \vdash e \Downarrow v \quad \textit{or} \quad \langle \mathscr{E}, e \rangle \Downarrow v$$

Idea. We keep track of their values in an *environment*

We evaluate *relative* to the environment and *lazily* fill in variable values along the way

Recall: Closures

$$(\mathscr{E}, e)$$

Def. *(informal)* A **closure** is an expression plus an environment

It *captures* bindings it needs to evaluate its expression

Idea. Functions need to *remember* what the environment looked like in order to behavior correctly (according to lexical scoping)

Recall: Named Closures

$$(\mathscr{E}, \mathsf{name} \mapsto \lambda x\,.\,e)$$

We need to be able to *name* closures

Idea. Named closures will put themselves into their environment *when they're called*

Recall: Lambda Calculus$^+$ (Syntax)

```
<expr> ::= λ<var>.<expr>
         | <var>
         | <expr><expr>
         | let <var> = <expr>
           in <expr>
         | <num>

<val>  ::= λ<var>.<expr>
         | <num>
```

Example (Dynamic)

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow \lambda x\,.\,e} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathscr{E}_2 = \mathscr{E}_1[x \mapsto v_1] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \mathsf{let}\ x = e_1\ \mathsf{in}\ e_2 \Downarrow v_2}$$

$$\frac{\mathscr{E} \vdash e_1 \Downarrow \lambda x\,.\,e \qquad \mathscr{E} \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}' = \mathscr{E}[x \mapsto v_2] \qquad \mathscr{E}' \vdash e \Downarrow v}{\mathscr{E} \vdash e_1 e_2 \Downarrow v}$$

$$\emptyset \ \vdash\ \mathsf{let}\ x = 0\ \mathsf{in}\ \mathsf{let}\ f = \lambda y\,.\,x\ \mathsf{in}\ \mathsf{let}\ x = 1\ \mathsf{in}\ f\ 0\ \Downarrow\ 1$$

Example (Lexical)

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)} \qquad \frac{}{\mathscr{E} \vdash n \Downarrow n} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow v_1 \qquad \mathscr{E}_2 = \mathscr{E}_1[x \mapsto v_1] \qquad \mathscr{E}_2 \vdash e_2 \Downarrow v_2}{\mathscr{E}_1 \vdash \text{let } x = e_1 \text{ in } e_2 \Downarrow v_2}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}$$

$$\emptyset \;\vdash\; \text{let } x = 0 \text{ in let } f = \lambda y\,.\,x \text{ in let } x = 1 \text{ in } f\ 0 \;\Downarrow\; 1$$

Practice Problem

```
let x = 0 in
let g = fun y -> x + 1 in
let x = 1 in
let f = fun y -> g x in
let x = 2 in
f
```

What (closure) does the following expression evaluate to? You don't need to give the derivation

Answer

```
let x = 0 in
let g = fun y -> x + 1 in
let x = 1 in
let f = fun y -> g x in
let x = 2 in
f
```

Type Theory

What is a Type?

```
let f : int -> int = ...
```

What is a Type?

```
let f : int -> int = ...
```

Who knows...

What is a Type?

```
let f : int -> int = ...
```

Who knows...

A **type** is an *syntactic object* given to an expression to describe its (semantic) behavior

What is a Type?

```
let f : int -> int = ...
```

Who knows...

A **type** is an *syntactic object* given to an expression to describe its (semantic) behavior

This description can be used to *restrict* the use of the expression *within* a program

What is a Type?

```
let f : int -> int = ...
```

Who knows...

A **type** is an *syntactic object* given to an expression to describe its (semantic) behavior

This description can be used to *restrict* the use of the expression *within* a program

Types help us delineate "well-behaved" programs

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

<u>The goal is to balance:</u>

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

<u>The goal is to balance:</u>

» Simplicity/Usability

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

<u>The goal is to balance:</u>

- Simplicity/Usability
- Expressivity

Trade-offs

$$(\lambda x\,.\,xx)(\lambda x\,.\,xx)$$

lambda term called Ω

Types are *restrictive*. They tells us what we *can't* do in our programs

But types are *safe*. They make sure we don't do dumb things in our program

<u>The goal is to balance:</u>

- Simplicity/Usability
- Expressivity
- Safety/Theoretical Guarantees

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml tells us when we try to write ill-behaved program

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml tells us when we try to write ill-behaved program

But OCaml also has strong *type inference* and *polymorphism*

OCaml

```
# let big_omega =
    let little_omega x = x x in
    little_omega little_omega;;
Error: This expression has type 'a -> 'b
       but an expression was expected of type 'a
       The type variable 'a occurs inside 'a -> 'b
```

OCaml tells us when we try to write ill-behaved program

But OCaml also has strong *type inference* and *polymorphism*

More expressive means more complex, designing PLs is finding the balance

Simply-Typed Lambda Calculus

Recall: Typing Judgments

$$\Gamma \vdash e : \tau$$

This judgment reads:

e has type τ in the context Γ

We say that e is **well-typed** if $\cdot \vdash e : \tau$ for some type τ

Recall: Typing Judgments

$$\Gamma \vdash e : \tau$$

This judgment reads:

e has type τ in the context Γ

We say that e is **well-typed** if $\cdot \vdash e : \tau$ for some type τ

Most of what type theorists do is come up with rules for deriving typing judgments

STLC Syntax

```
<e>  ::= () | <v> | <e> <e>
       | fun ( <v> : <ty> ) -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

STLC Syntax

```
<e>  ::= () | <v> | <e> <e>
       | fun ( <v> : <ty> ) -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

The syntax is the same as that of the lambda calculus except:

STLC Syntax

```
<e>  ::= () | <v> | <e> <e>
       | fun ( <v> : <ty> ) -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

The syntax is the same as that of the lambda calculus except:

- we include a unit expression

STLC Syntax

```
<e>  ::= () | <v> | <e> <e>
       | fun ( <v> : <ty> ) -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

The syntax is the same as that of the lambda calculus except:

- we include a unit expression
- we have types, which annotate arguments

STLC Syntax

```
<e>  ::= () | <v> | <e> <e>
       | fun ( <v> : <ty> ) -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

The syntax is the same as that of the lambda calculus except:

- we include a unit expression
- we have types, which annotate arguments

This is the first time that **types are a part of our syntax**

Syntax

$$e ::= \bullet \mid x \mid \lambda x^{\tau}.\, e \mid ee$$

$$\tau ::= \top \mid \tau \to \tau$$

The syntax is the same as that of the lambda calculus except:

- » we include a unit expression
- » we have types, which annotate arguments

This is the first time that **types are a part of our syntax**

STLC Typing

STLC Typing

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

STLC Typing

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

STLC Typing

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \texttt{abstraction}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

STLC Typing

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \texttt{abstraction}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \texttt{application}$$

STLC Typing

$$\frac{}{\Gamma \vdash \bullet : \top}\ \texttt{unit}$$

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}\ \texttt{abstraction}$$

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau}\ \texttt{variable}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}\ \texttt{application}$$

These rules enforce that a function can only be applied if we *know* that it's a function

Type Annotations?

```
<e>  ::= () | <v> | <e> <e>
       | fun <v> -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x . e : \tau \to \tau'}$$

Type Annotations?

```
<e>  ::= () | <v> | <e> <e>
       | fun <v> -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x . e : \tau \to \tau'}$$

Do we have to include the type annotation on function arguments?

Type Annotations?

```
<e>  ::= () | <v> | <e> <e>
       | fun <v> -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x . e : \tau \to \tau'}$$

Do we have to include the type annotation on function arguments?

No, but it does change the way typing works

Type Annotations?

```
<e>  ::= () | <v> | <e> <e>
       | fun <v> -> <e>
<ty> ::= unit | <ty> -> <ty>
<v>  ::= a | ... | z
```

$$\frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x . e : \tau \to \tau'}$$

Do we have to include the type annotation on function arguments?

No, but it does change the way typing works

If we include annotations we're using **Church-style typing.** If we drop annotations, we're using **Curry-style typing**

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

In **Curry-style typing,** the type of an expression is *extrinsic*, the expression is just an expression in the lambda calculus

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

In **Curry-style typing**, the type of an expression is *extrinsic*, the expression is just an expression in the lambda calculus

In **Church-style typing**, it's *intrinsic*, built into the expression and the semantics

Aside: Church vs. Curry Typing

```
fun x -> x

fun (x : unit) -> x
```

What is the type of the first expression? How about the second?

In **Curry-style typing**, the type of an expression is *extrinsic*, the expression is just an expression in the lambda calculus

In **Church-style typing**, it's *intrinsic*, built into the expression and the semantics

(using Curry-style typing is not the same as having polymorphism)

Uniqueness of Types

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Proof. The rough idea is to do induction *on the derivations themselves* (whoa)

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Proof. The rough idea is to do induction *on the derivations themselves* (whoa)

Every expression has a *unique type* in STLC with Church-style typing

Uniqueness of Types

Lemma. If $\Gamma \vdash e : \tau_1$ and $\Gamma \vdash e : \tau_2$ then $\tau_1 = \tau_2$

Proof. The rough idea is to do induction *on the derivations themselves* (whoa)

Every expression has a *unique type* in STLC with Church-style typing

In particular, the function `type_of` is well-defined

STLC Semantics (Review)

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)}\ \texttt{fun}$$

$$\frac{}{\mathscr{E} \vdash \bullet \Downarrow \bullet}\ \texttt{unit}$$

$$\frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}\ \texttt{variable}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}\ \texttt{application}$$

STLC Semantics (Review)

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)}\ \texttt{fun} \qquad \frac{}{\mathscr{E} \vdash \bullet \Downarrow \bullet}\ \texttt{unit} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}\ \texttt{variable}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}\ \texttt{application}$$

The semantics are *identical*

STLC Semantics (Review)

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)}\ \texttt{fun} \qquad \frac{}{\mathscr{E} \vdash \bullet \Downarrow \bullet}\ \texttt{unit} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}\ \texttt{variable}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}\ \texttt{application}$$

The semantics are *identical*

This is part of the point: Type-checking only determines *whether* we evaluate the program (whether it makes sense to)

STLC Semantics (Review)

$$\frac{}{\mathscr{E} \vdash \lambda x\,.\,e \Downarrow (\mathscr{E}, \lambda x\,.\,e)}\ \texttt{fun} \qquad \frac{}{\mathscr{E} \vdash \bullet \Downarrow \bullet}\ \texttt{unit} \qquad \frac{\mathscr{E}(x) = v}{\mathscr{E} \vdash x \Downarrow v}\ \texttt{variable}$$

$$\frac{\mathscr{E}_1 \vdash e_1 \Downarrow (\mathscr{E}_2, \lambda x\,.\,e) \qquad \mathscr{E}_1 \vdash e_2 \Downarrow v_2 \qquad \mathscr{E}_3 = \mathscr{E}_2[x \mapsto v_2] \qquad \mathscr{E}_3 \vdash e \Downarrow v}{\mathscr{E}_1 \vdash e_1 e_2 \Downarrow v}\ \texttt{application}$$

The semantics are *identical*

This is part of the point: Type-checking only determines *whether* we evaluate the program (whether it makes sense to)

It doesn't determine **how** we evaluate the program

Example

$$\lambda x^{\tau}.xx$$

What happens if we try to give a type to the above expression? What should τ be?

Practice Problem

$$\frac{(x : \tau) \in \Gamma}{\Gamma \vdash x : \tau} \qquad \frac{\Gamma, x : \tau \vdash e : \tau'}{\Gamma \vdash \lambda x^{\tau}.\, e : \tau \to \tau'}$$

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

$\cdot \vdash \lambda f^{\top \to \top}.\, \lambda x^{\top}.\, fx : (\top \to \top) \to \top \to \top$

Give a derivation for the above judgment

Answer

$$\cdot \vdash \lambda f^{\top \to \top}.\lambda x^{\top}.fx : (\top \to \top) \to \top \to \top$$

How do we know if we've defined a "good" programming language?

Type Safety

Type Safety

Thm. If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

With small-step semantics, we can give a finer-grained analysis:

> **Thm.** If $\cdot \vdash e : \tau$, then
>
> » *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$
>
> » *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

Type Safety

> **Thm.** If $\cdot \vdash e : \tau$ and there is a value v such that $\langle \varnothing, e \rangle \Downarrow v$ then $\cdot \vdash e(v) : \tau$

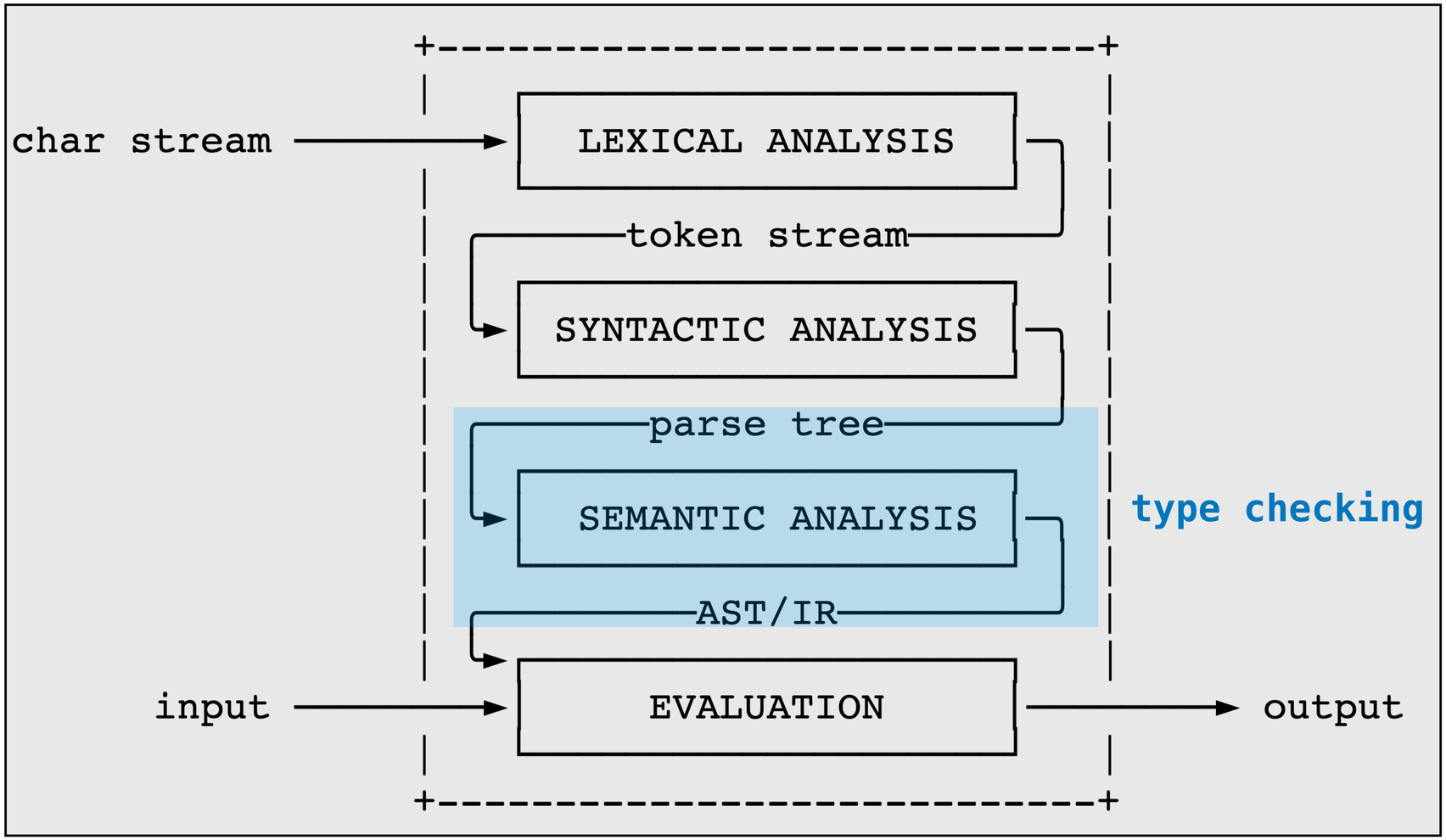
With small-step semantics, we can give a finer-grained analysis:

> **Thm.** If $\cdot \vdash e : \tau$, then
>
> » *(progress)* either e is a value or there is an e' such that $e \longrightarrow e'$
>
> » *(preservation)* If $\cdot \vdash e : \tau$ and $e \longrightarrow e'$ then $\cdot \vdash e' : \tau$

These results are *fundamental*. They tell us that our PL is well-behaved (it's a "good" PL)

Type Checking

The Picture


char stream
LEXICAL ANALYSIS
token stream
SYNTACTIC ANALYSIS
parse tree
SEMANTIC ANALYSIS
type checking
AST/IR
input
EVALUATION
output

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type inference is the problem of *synthesizing* a type for a given expression, if possible

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type inference is the problem of *synthesizing* a type for a given expression, if possible

Theoretically, these two problems can be very different

Type Checking vs. Type Inference

```
type_check : expr -> ty -> bool

type_of : expr -> ty option
```

Type checking the problem of determining whether a given expression is a given type

Type inference is the problem of *synthesizing* a type for a given expression, if possible

Theoretically, these two problems can be very different

For STLC, they are both easy

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

It seems like we need to do *some* amount of inference because it's not immediately clear what type we should check e_1 to be

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

It seems like we need to do *some* amount of inference because it's not immediately clear what type we should check e_1 to be

Aside: If you're interested there is a way of *combining* checking and inference in what's called *bidirectional type checking*

The One Issue

$$\frac{\Gamma \vdash e_1 : \tau \to \tau' \qquad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 e_2 : \tau'}$$

How do we turn this into a type-checking procedure?

It seems like we need to do *some* amount of inference because it's not immediately clear what type we should check e_1 to be

Aside: If you're interested there is a way of *combining* checking and inference in what's called *bidirectional type checking*

Our solution: We'll just use type inference

demo

(STLC)

Summary

Type systems delineate well-behaved expressions